

EMBEDDED SYSTEM WITH IOT

EXPERIMENT NO. 4

IoT based Irrigation System

Name: NAVARRO, ROD GERYK C.

Course/Section: CPE162P-4/E01

Date of Performance: 05/19/2025

Date of Submission: 05/23/2025

CYREL O. MANLISES, PH.D.

Instructor

DISCUSSION

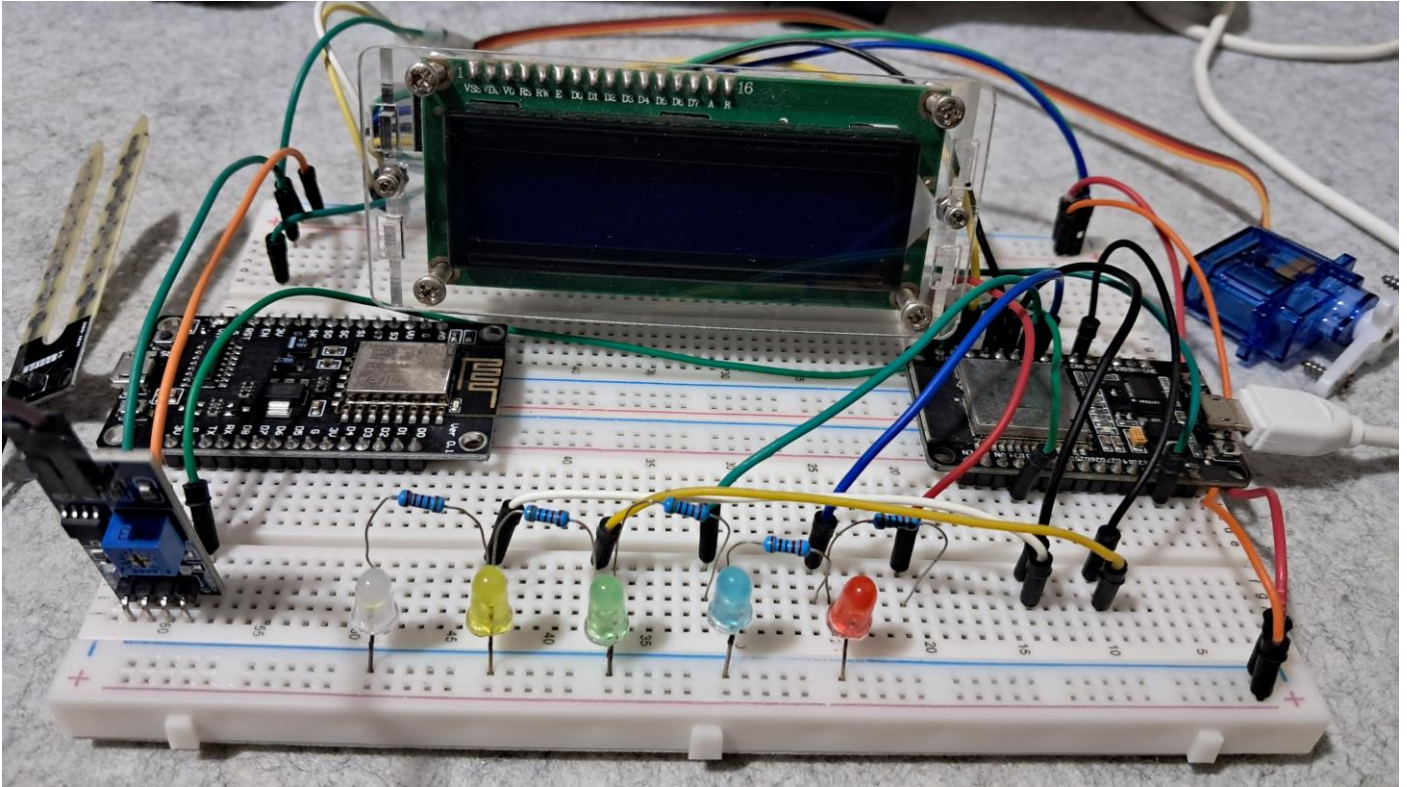


FIGURE 1: The Connection of the Moisture Sensor, Servo Motor, LEDs, and LCD to the ESP32 Dev Module

Connections

In this experiment, I created an IoT-based irrigation system designed to water plants at home. An irrigation system is made up of parts that work together to deliver water to plants in a controlled way. I used several components for this setup: a moisture sensor to measure the soil's moisture level, a servo motor that acts as a water pump, and five colored LEDs (white for dry, yellow for low, green for moderate, blue for high, and red for saturated) to show the moisture status visually. An LCD screen displays both the moisture level and the raw data from the sensor. The ESP32 Dev Module is the main controller of the system. Since ESP32 has built-in Wi-Fi, I was able to send data to the internet.

The first step was connecting all the parts to the ESP32. For monitoring and analyzing the data, I used ThingSpeak, an IoT platform that displays real-time updates. The ESP32 sends data from the moisture sensor and the servo motor's status to ThingSpeak, allowing me to monitor the condition of the soil and the system's operation online. After this, I programmed the system using the Arduino IDE to make all the components work together properly.

Code Development

```
Navarro_EXP4_CPE162P.ino
1  #include <LiquidCrystal_I2C.h>
2  #include <WiFi.h>
3  #include <ThingSpeak.h>
4  #include <ESP32Servo.h>
5
6  // WiFi Configuration
7  const char* ssid = "PLDTHOMEFIBRT6ucX";
8  const char* password = "PLDTWIFITLK72";
9
10 // ThingSpeak Configuration
11 const long myChannelNumber = 2964917 ;
12 const char myWriteAPIKey[] = "18A6ZBU4I30LZZ44";
13
14 //hardware configuration
15 const int MOISTURE_PIN = 34;
16 const int WHITE_LED = 19;
17 const int YELLOW_LED = 4;
18 const int GREEN_LED = 18;
19 const int BLUE_LED = 5;
20 const int RED_LED = 23;
21 const int SERVO_PIN = 13;
22
23 //default calibration values (4095 for dry and 1550 for saturated)
24 int dryValue = 4095;
25 int saturatedValue = 1550;
26 const int NUM_READINGS = 50;
27
28 //LCD and servo
29 LiquidCrystal_I2C lcd(0x27, 16, 2);
30 Servo irrigationServo;
31 int servoPosition = 0;
32
33 // system parameters
34 const int UPDATE_INTERVAL = 5000;
35 unsigned long lastUpdate = 0;
36 WiFiClient client;
37 bool servoActivated = false;
```

FIGURE 2: The Initialization of Variables and Components

In this part of the code, I started by including the needed libraries like LiquidCrystal_I2C, WiFi, ThingSpeak, and ESP32Servo, which are required to make the LCD, internet connection, and servo motor work. Then, I defined the Wi-Fi name and password to connect the ESP32 to the internet, and I set up the ThingSpeak channel number and API key so the ESP32 can send data online. After that, I assigned the pins for all the components, including the moisture sensor, LEDs, and the servo motor. I also set some default values for calibration, like the dry and saturated soil readings, and created variables to handle the LCD display, servo motor position, update timing, and a flag to check if the servo was activated or not.

```
Navarro_EXP4_CPE162P.ino
39
40 void setup() {
41     initializePins();
42     initializeDisplay();
43     initializeSerial();
44     showStartupScreen();
45     calibrateSensor();
46     connectToWiFi();
47     ThingSpeak.begin(client);
48 }
49
```

FIGURE 3: The Setup of the Components

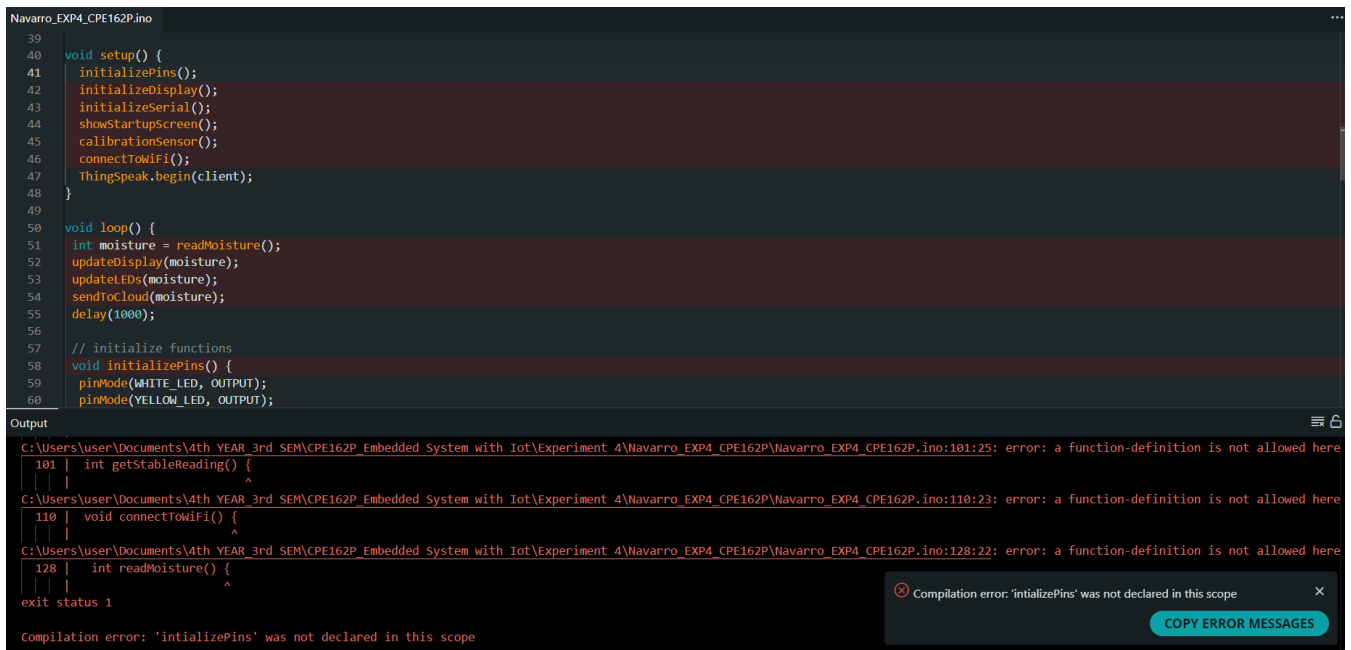
In the setup() function, I prepared everything needed before the system runs. I called several functions to initialize pins (set the LED and servo pins as outputs), start the LCD display, and begin the serial monitor for debugging. I also showed a welcome message on the LCD and ran the calibration function to get average moisture readings for dry and wet soil. Then, I connected the ESP32 to the Wi-Fi using the provided SSID and password, and once connected, the IP address was shown on the LCD. Finally, I started ThingSpeak so it would be ready to receive data from the ESP32.

```
Navarro_EXP4_CPE162P.ino
50 void loop() {
51   int moisture = readMoisture();
52   updateDisplay(moisture);
53   updateLEDs(moisture);
54   sendToCloud(moisture);
55   delay(1000);
56 }
```

FIGURE 4: The Loop Function

In the loop() function, which runs repeatedly, the system reads the current soil moisture using the sensor. After getting the moisture value, it updates the LCD to show the raw value and the moisture level as text (like "Dry", "Low", or "Moderate"). It also checks which LED should light up based on the moisture level, and whether to turn the servo motor on or off depending on how dry the soil is. Lastly, it sends the moisture data and the servo status to ThingSpeak every few seconds, so I can monitor the plant's condition online. A short delay is added at the end to slow down the loop and avoid sending too much data at once.

Errors Encountered during Code Development



The screenshot shows an IDE with a code editor and an output window. The code editor displays the following code:

```
39
40 void setup() {
41   initializePins();
42   initializeDisplay();
43   initializeSerial();
44   showStartupScreen();
45   calibrationSensor();
46   connectToWiFi();
47   Thingspeak.begin(client);
48 }
49
50 void loop() {
51   int moisture = readMoisture();
52   updateDisplay(moisture);
53   updateLEDs(moisture);
54   sendToCloud(moisture);
55   delay(1000);
56
57   // initialize functions
58   void initializePins() {
59     pinMode(WHITE_LED, OUTPUT);
60     pinMode(YELLOW_LED, OUTPUT);
61   }
62 }
```

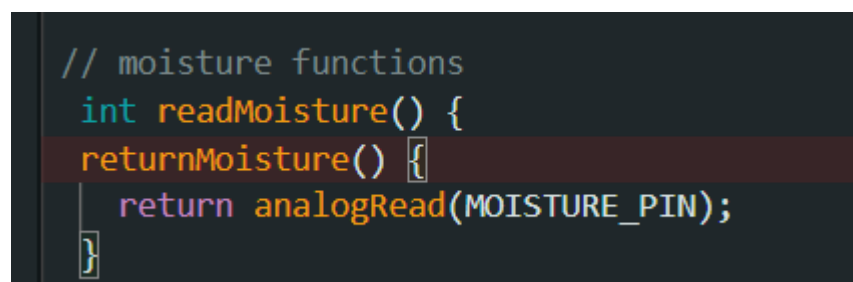
The output window shows the following error messages:

```
C:\Users\user\Documents\4th YEAR 3rd SEM\CPE162P Embedded System with Iot\Experiment 4\Navarro_EXP4_CPE162P\Navarro_EXP4_CPE162P.ino:101:25: error: a function-definition is not allowed here
101 | int getStableReading() {
    | ^
C:\Users\user\Documents\4th YEAR 3rd SEM\CPE162P Embedded System with Iot\Experiment 4\Navarro_EXP4_CPE162P\Navarro_EXP4_CPE162P.ino:110:23: error: a function-definition is not allowed here
110 | void connectToWiFi() {
    | ^
C:\Users\user\Documents\4th YEAR 3rd SEM\CPE162P Embedded System with Iot\Experiment 4\Navarro_EXP4_CPE162P\Navarro_EXP4_CPE162P.ino:128:22: error: a function-definition is not allowed here
128 | int readMoisture() {
    | ^
exit status 1
Compilation error: 'intializePins' was not declared in this scope
```

A tooltip message is also visible: "Compilation error: 'intializePins' was not declared in this scope".

FIGURE 5: Error 1

As shown on Figure 5 is the error that I first encountered during code development, it is a result of many misspelled names, and wrong closure of functions. This error doesn't cause me a lot of time to fix since it is very easy to identify the root cause of the problem.



The screenshot shows a code editor with the following code:

```
// moisture functions
int readMoisture() {
  returnMoisture() {
    return analogRead(MOISTURE_PIN);
  }
}
```

FIGURE 6: Error 2

The image shown in Figure 6 is another error that I have faced when fixing the closure of each function. As we can see on the image the error line has its own curly bracket that resulted to a confusion on the closure of each function.

Difficult Aspect of the Experiment's Development

The most difficult part of the experiment was figuring out the cause of the inconsistent readings of my moisture sensor. The readings don't change even though I exposed the sensor to various moisture levels. I tried multiple ways to fix it such as recalibrating, changing power source, and changing the code. However, the only solution that worked was changing the moisture sensor. After changing the sensor, I never received any incorrect readings from the sensor.

Interpretations of ThingSpeak Visualization

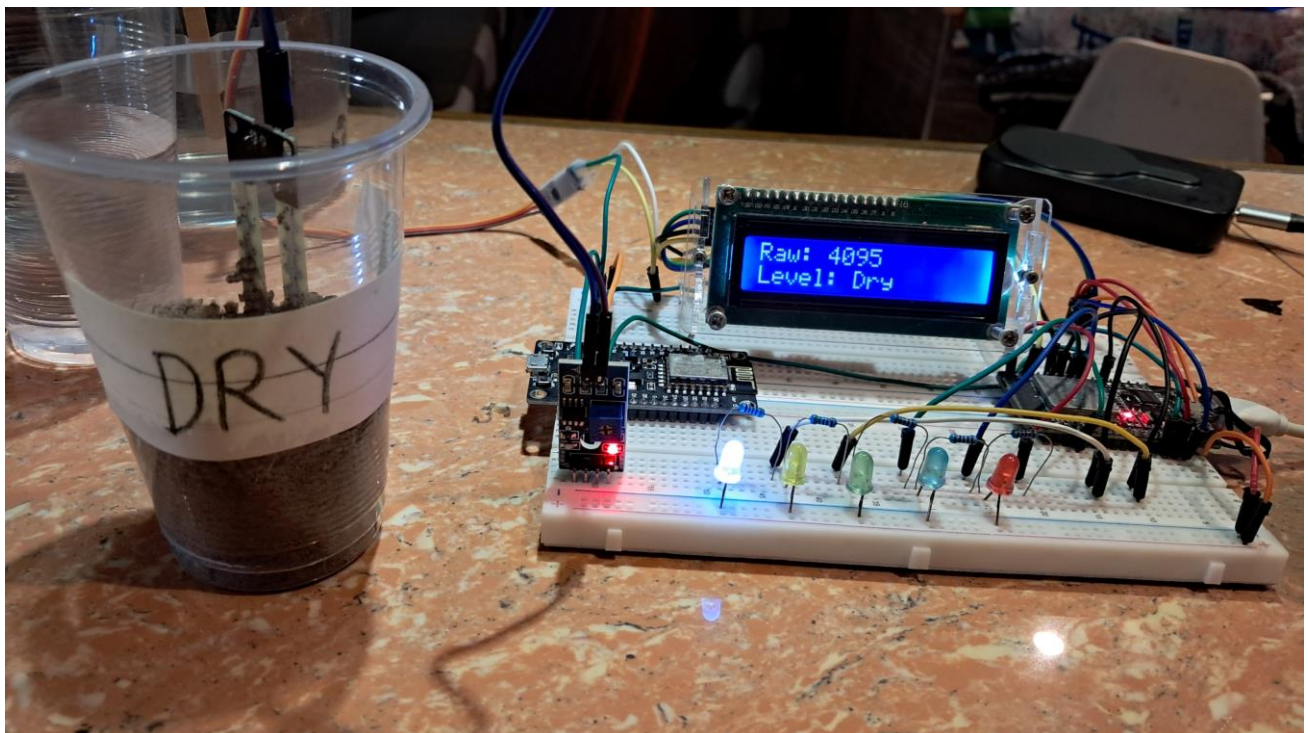
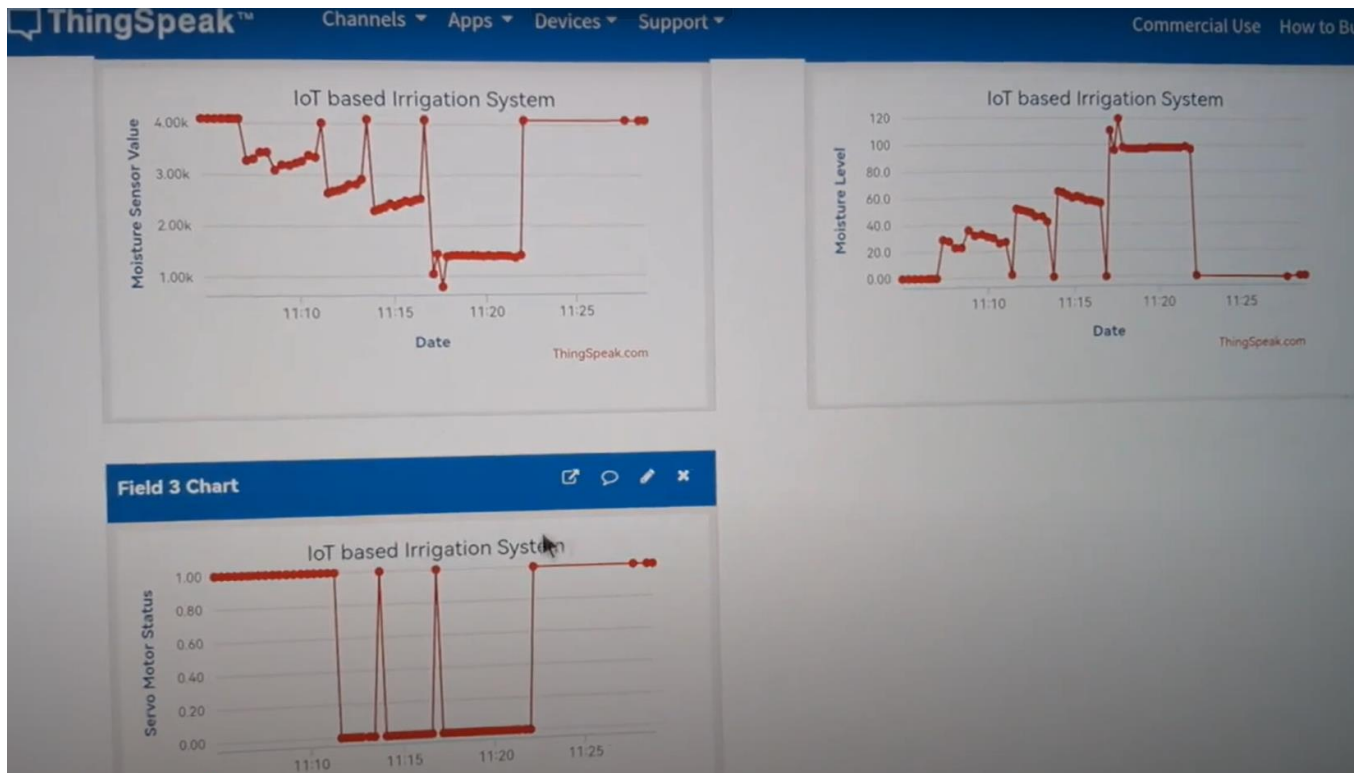


FIGURE 7: ThingSpeak Visualization: Dry Moisture Level

In the image above, I placed the moisture sensor in a dry soil. On the LCD screen we can see that the soil moisture level was at the dry level, as well as the raw value of the sensor data which is in the range of a dry soil moisture. From the ThingSpeak we can visualized the graph of the servo motor status, which has a value of 1, which means the servo motor is turned ON to deliver water to the soil. The servo motor will keep its motion until the soil reaches the required moisture level.

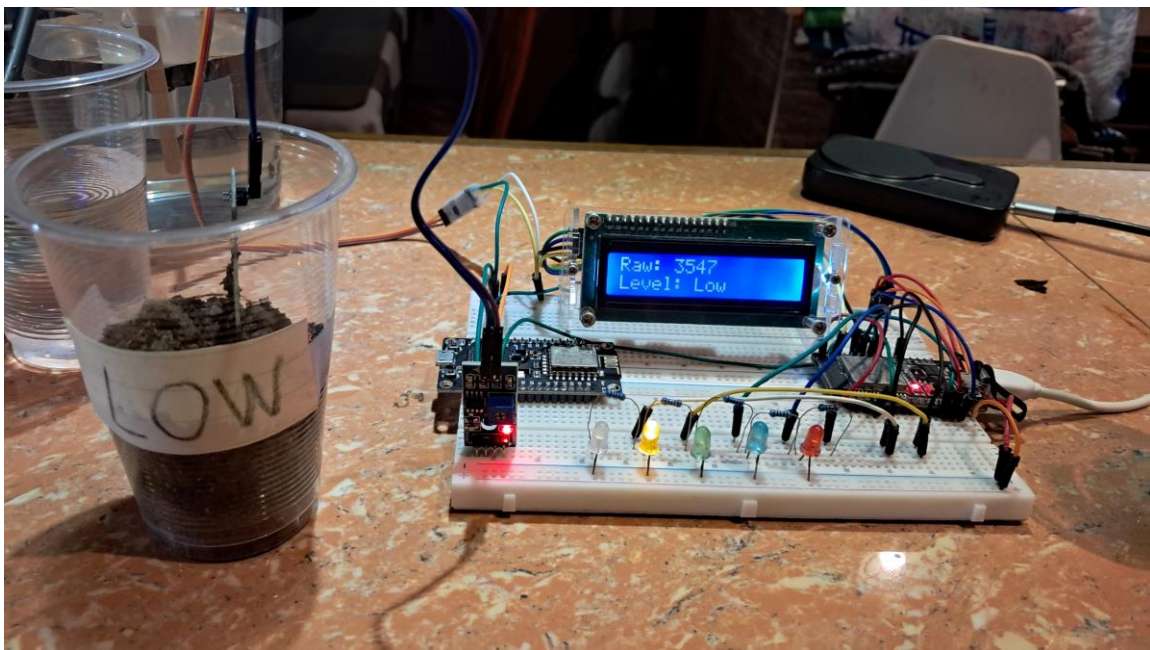
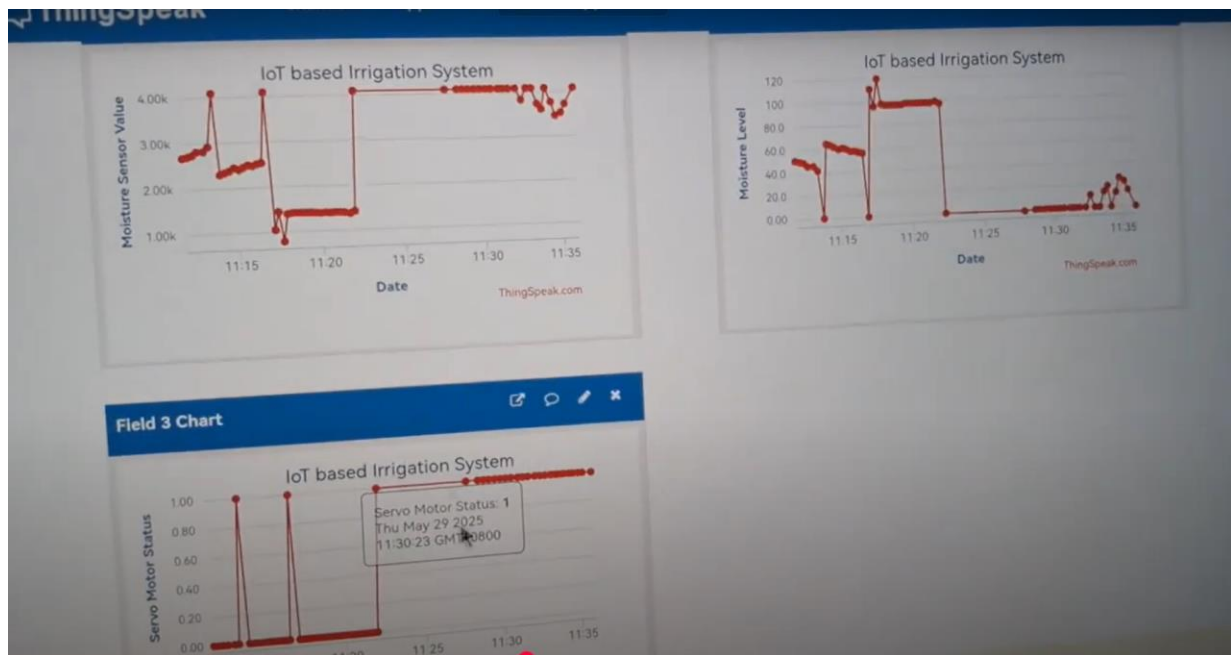


FIGURE 8: ThingSpeak Visualization: Low Moisture Level

When we look at the graph of the moisture sensor value and the moisture level, even though I placed the sensor in a low soil moisture it gives an unstable reading from the sensor. After reaching the low moisture level it goes back to a dry soil moisture level. At this state the servo motor is turned ON since it hasn't reached the target soil moisture level, that's why it remains to have a value of 1. Also, the yellow LED lights up to indicate a low soil moisture.

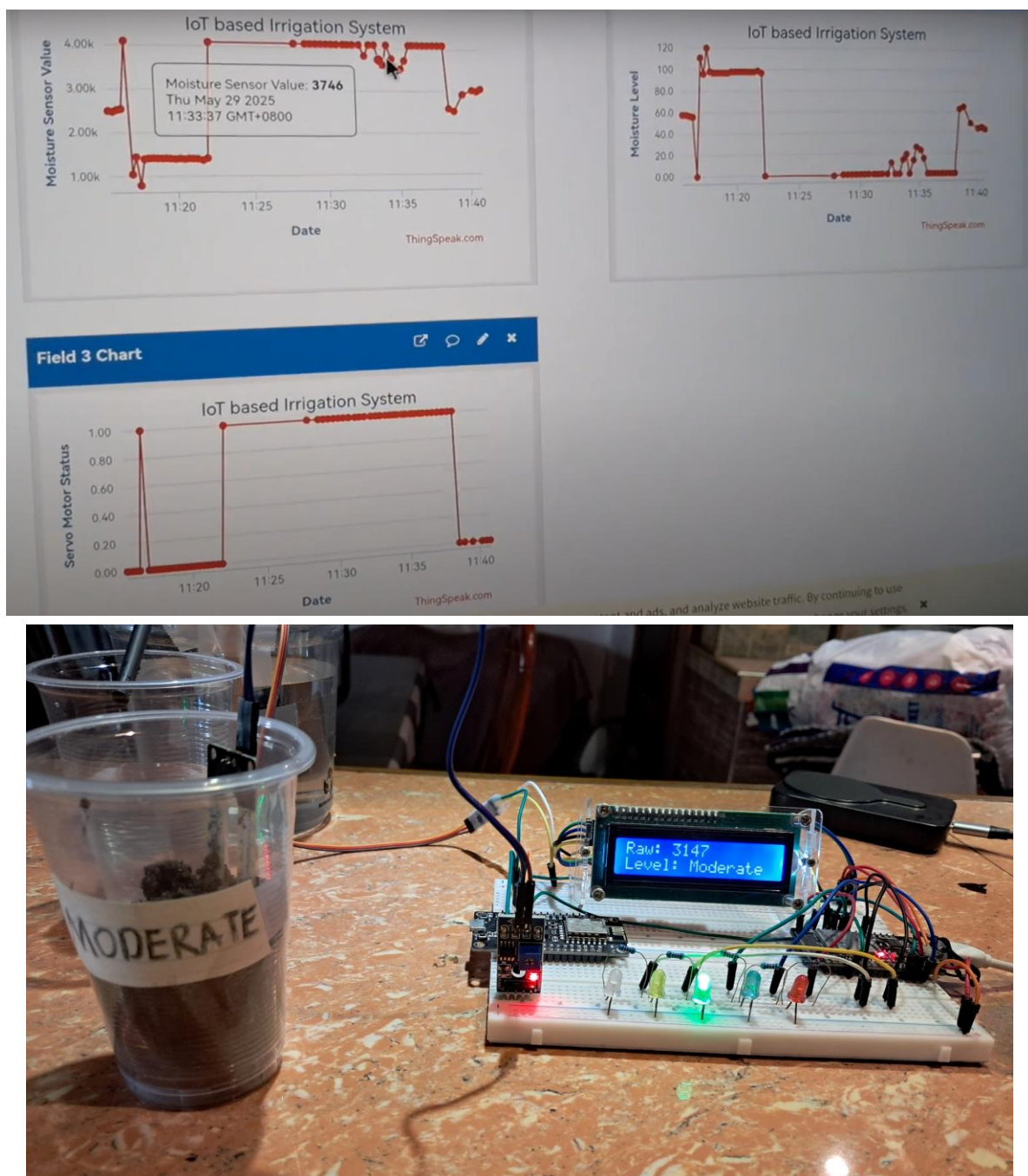


FIGURE 9: ThingSpeak Visualization: Moderate Moisture Level

At this level of moisture, the servo motor stops to deliver water to the soil since it already reaches the right amount of moisture in the soil. The servo motor has now a value of 0 which means turned OFF. Also, the green LED lights up to indicate that the soil reached the target moisture.

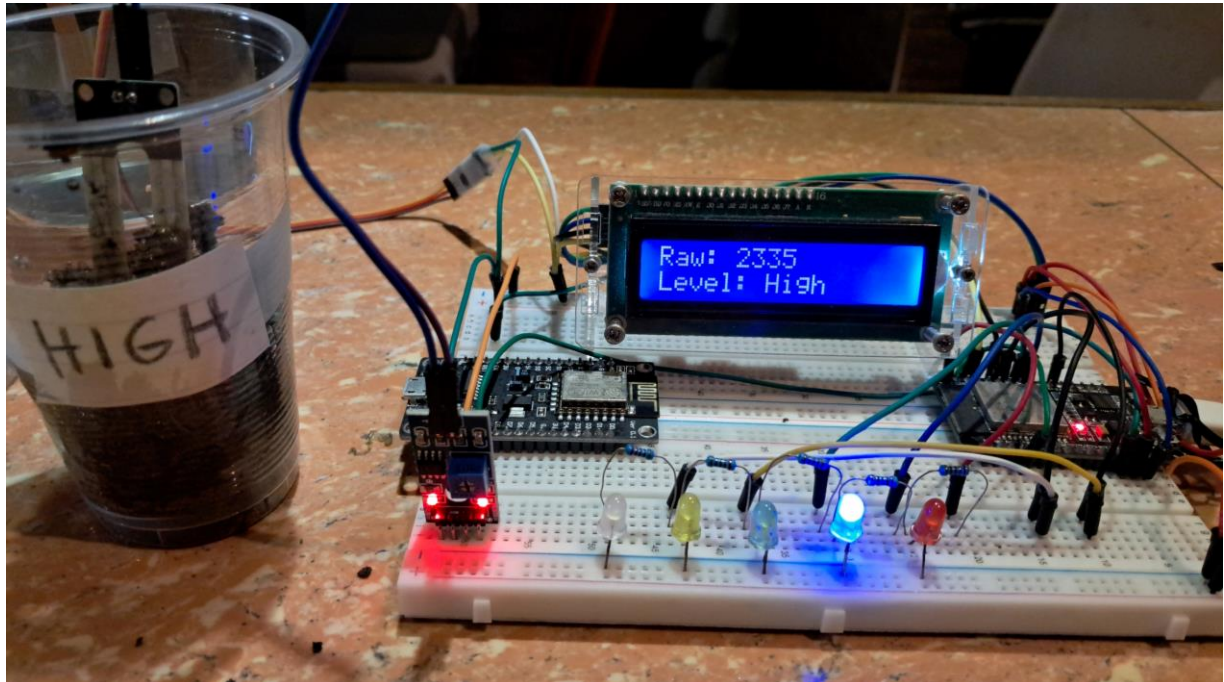


FIGURE 10: ThingSpeak Visualization: High Moisture Level

The moisture at this state exceeds the target moisture of the soil. The blue LED is turned ON to indicate a high soil moisture. As we can see on the graph that the higher the moisture level the lower the moisture sensor value. The servo motor has a value of 0 since the soil doesn't need water anymore. Even though the soil has a lot of moisture, it is not a good moisture for the plants.

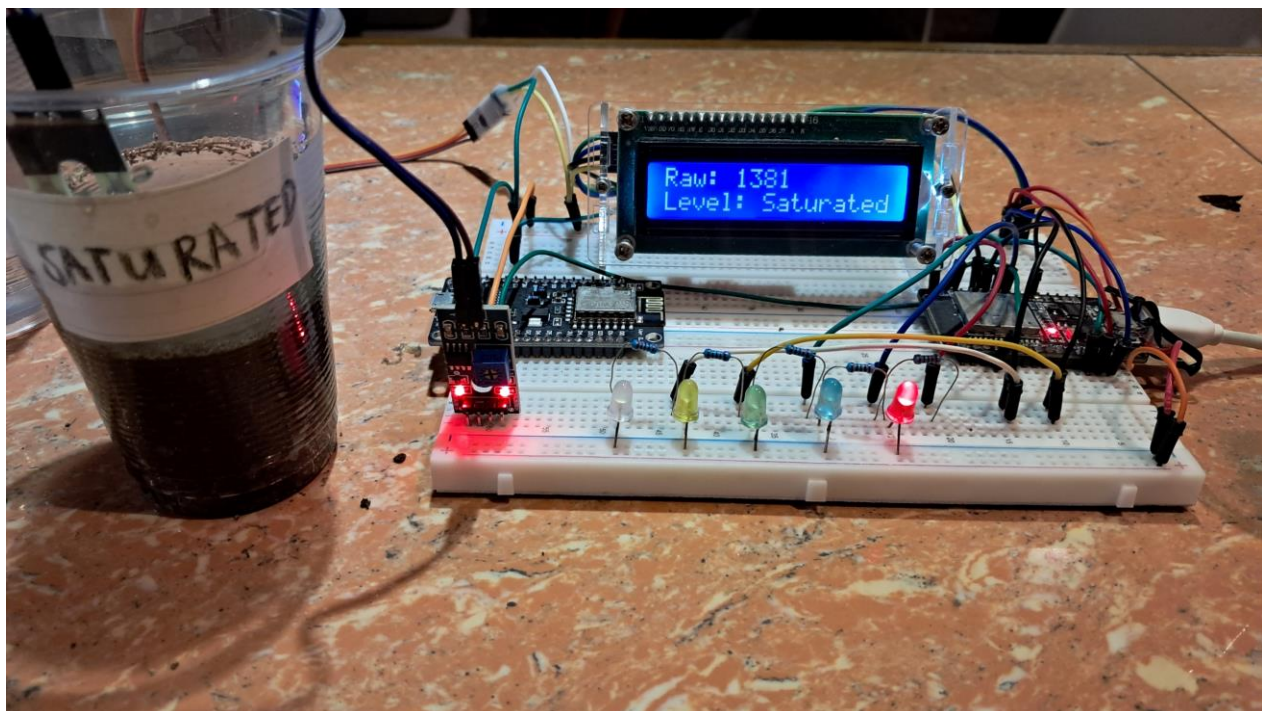
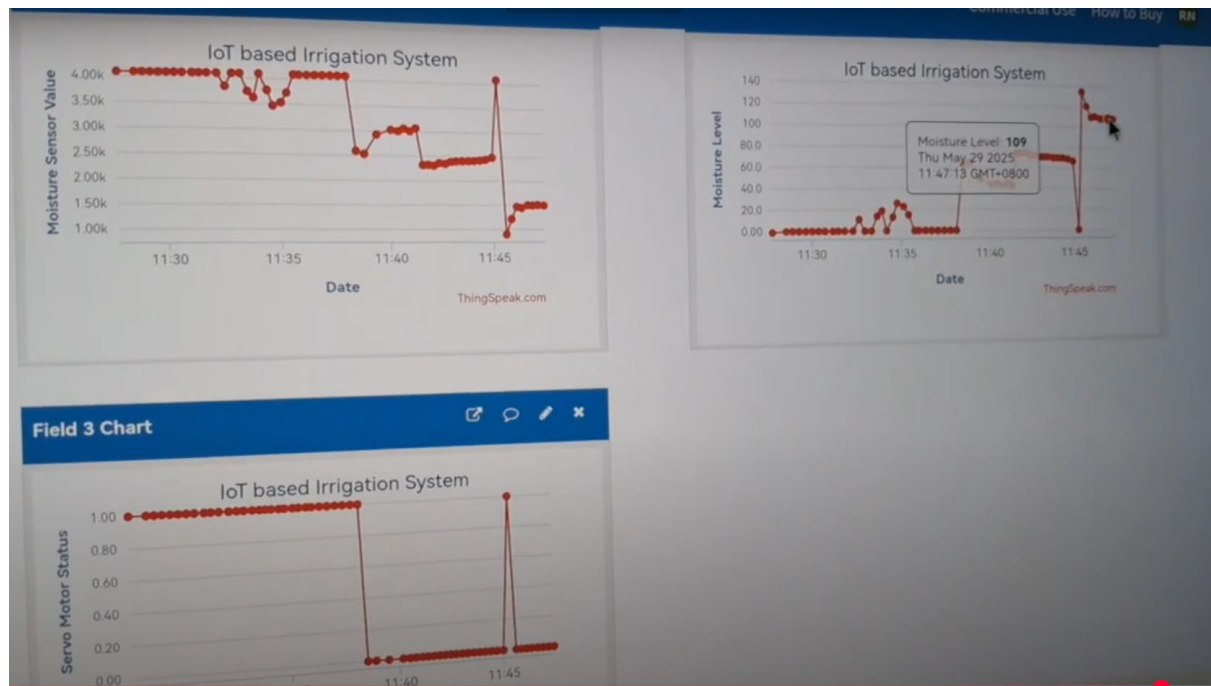


FIGURE 11: ThingSpeak Visualization: Saturated Moisture Level

In the graph, we can see that the soil reaches the highest level of moisture which is the saturated level. The moisture sensor value goes below 2000 that means it is in the range of a saturated soil moisture. This moisture is dangerous for the plants. We can expect that the servo motor is turned OFF since no water is needed for the soil. Also, the red LED is turned ON to indicate that the soil is at the saturated level of moisture.

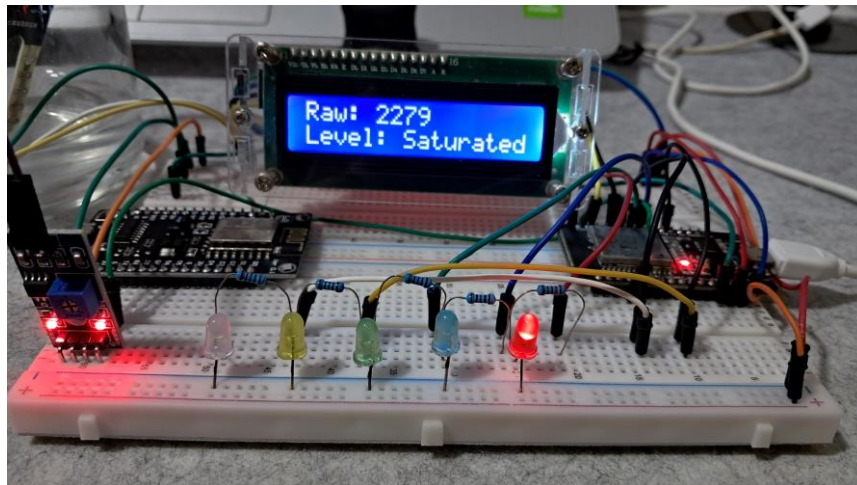


FIGURE 12: Working Prototype

This is the final working prototype of the IoT based irrigation system experiment, which successfully meets all the requirements.

CONCLUSION

In conclusion, this experiment gave me a hands-on experience in building a fully functional IoT-based irrigation system, which is designed to help automate plant watering at home. Through this project, I was able to understand how to integrate different components such as a moisture sensor, servo motor, LCD screen, and LEDs with the ESP32 Dev Module. Each part had a specific purpose the moisture sensor read the soil condition, the servo motor acted like a pump to deliver water, the LEDs gave a quick visual indicator of moisture levels, and the LCD showed live sensor data. I also used ThingSpeak, a cloud-based IoT platform, to visualize the moisture and motor status online. This allowed me to monitor the system from anywhere, making it more practical and user-friendly. It was satisfying to see how all the components worked together when the soil was dry, the motor turned on, and when the soil had enough moisture, the motor turned off automatically. The LEDs also changed color according to moisture levels, providing a fast way to check the plant's condition without needing to read the display.

Throughout the project, I encountered a few problems, especially during the code development and testing phases. One of the biggest challenges was identifying why the sensor was giving incorrect or unchanging readings, even when I exposed it to different moisture levels. I tried recalibrating, adjusting power sources, and editing the code, but nothing worked until I decided to replace the moisture sensor altogether which finally solved the issue. I also faced coding errors, like wrong function closures and misspelled variable names, but these were easier to fix once I carefully reviewed the error messages. After getting past those obstacles, I focused on making the program run smoothly. I used the Arduino IDE to set up the system logic, such as determining what happens at each moisture level and when the motor should activate. The ThingSpeak graphs were especially useful for understanding how the sensor readings

changed over time and how the servo responded accordingly. In the end, my working prototype successfully met all the project goals. It not only automated the watering process but also gave me a better understanding of how IoT systems work in real life. All in all, this experience has taught me valuable lessons in both electronics and programming, and it made me realize how technology can be used to solve everyday problems in a smart and efficient way.